Essentials of Ethical Hacking

- Lecture By
  Pratidnya S. Hegde Patil
  Assistant Professor-IT Dept.

Initial Activity (For Labs)

- Work on Firefox Browser
- Register yourself (sign-in)

  https://portswigger.net/web-security

  For Course: Web Security Academy (free of cost).
- Work on Burp suite Community Version.

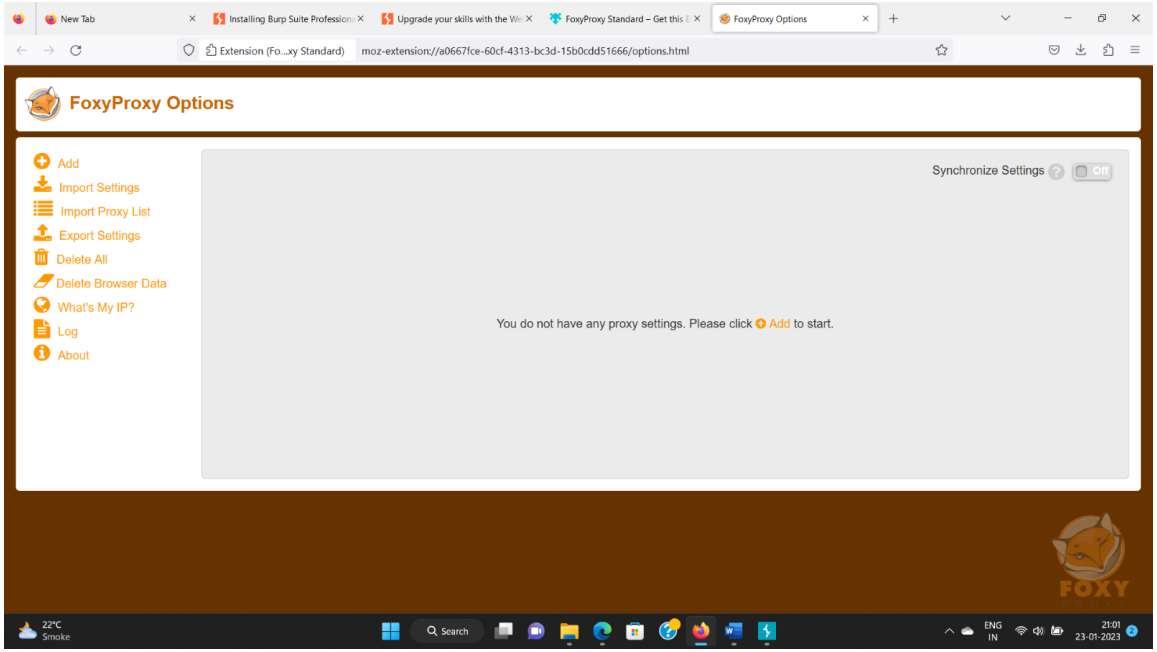
Installation

- Installation of Burp Suite Community Edition and settings:
  - Download Burp Suite Community Edition and install it.

https://portswigger.net/burp/documentation/desktop/getting-started/download-and-install

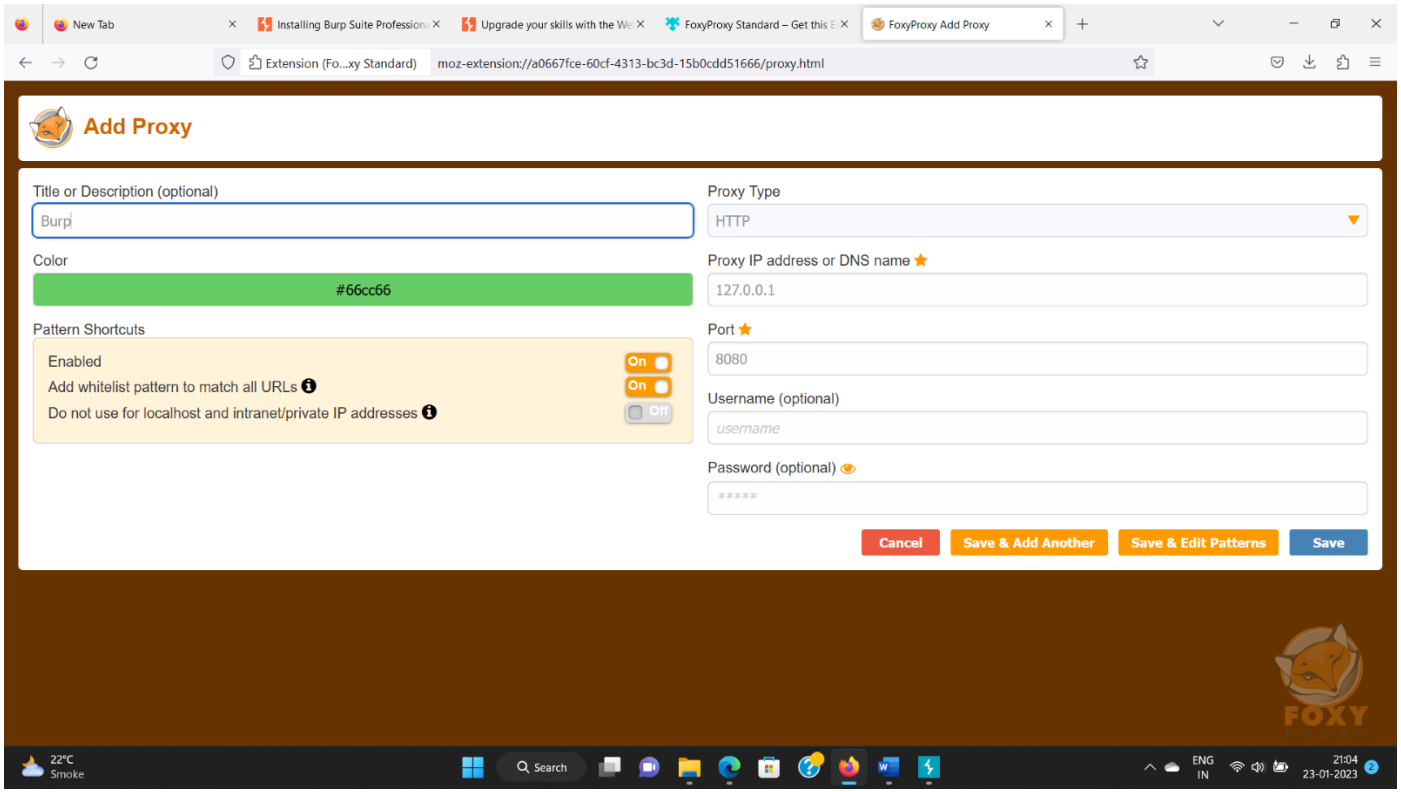
Installation

- **Download FireFox.**
- **Open FireFox and select extensions choose Foxy Proxy Extension. To configure it. Select Options button in Foxy Proxy.**



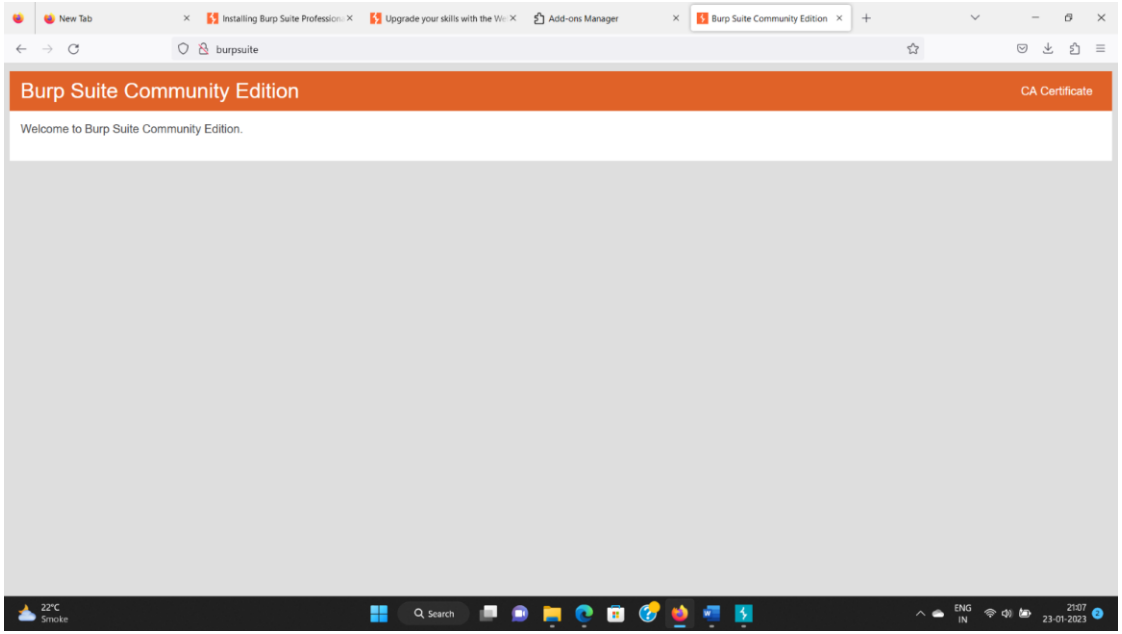
FireFox Burp Setting

- **Settings in Firefox:**



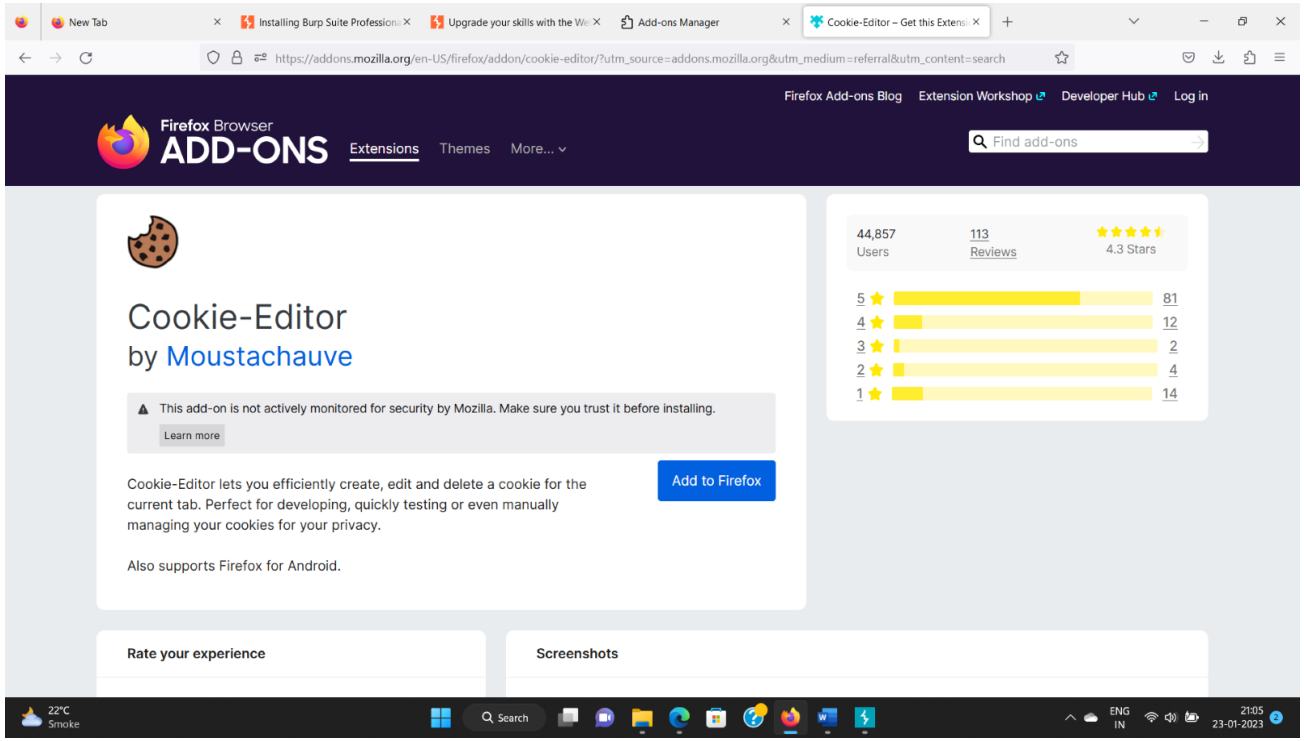
Certificate of BurpSuite

- Select Burp in Foxy Proxy Extension on FireFox. Select http://burpsuite link on firefox and select the Certificate to be installed.



FireFox Extension (Cookie Editor)

- Download Cookie Editor from Firefox Extensions.



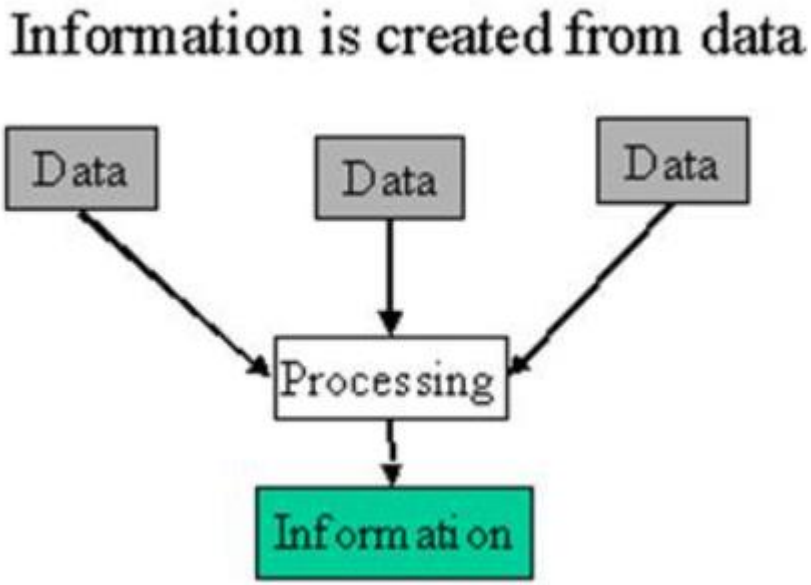
Information Security

Courtesy: EC Council EHE Course
https://codered.eccouncil.org/courseVideo/ethical-hacking-essentials

Pratidnya S. Hegde Patil

What is Information?

- Information is a representation or communication of knowledge, such as facts, data, or opinions, that can be conveyed through various mediums.



Information can be used to learn about the world and make decisions.
Ex: a weather report provides information about the temperature and chance of rain.

Why secure Information ?

- Information is a critical asset that organizations must secure. If an organization's sensitive information falls into the wrong hands, the organization may suffer considerable losses in terms of finances, brand reputation, or customers, or in other ways.

- Needs to be secured -> Information Security required.

What is Information Security?

- Is a state of well-being of information and infrastructure in which the possibility of theft, tampering, and disruption of information and services is low or tolerable.

- Refers to the protection or safeguarding of information and information systems that use, store, and transmit information from unauthorized access, disclosure, alteration, and destruction.

Need for Information Security

- Evolution of technology leading to ease of use.
- Relying on computers for accessing, providing or storing information.
- Increased network environment and network-based applications.
- Direct impact of security breach on the corporate asset base and goodwill.
- Increased complexity of computer infrastructure administration and management.

Motives, Goals and Objectives of Information Security Attacks

Attacks = Motive (Goal) + Method + Vulnerability

A motive originates out of the notion that the **target system stores or processes** something valuable, and this leads to the threat of an attack on the system

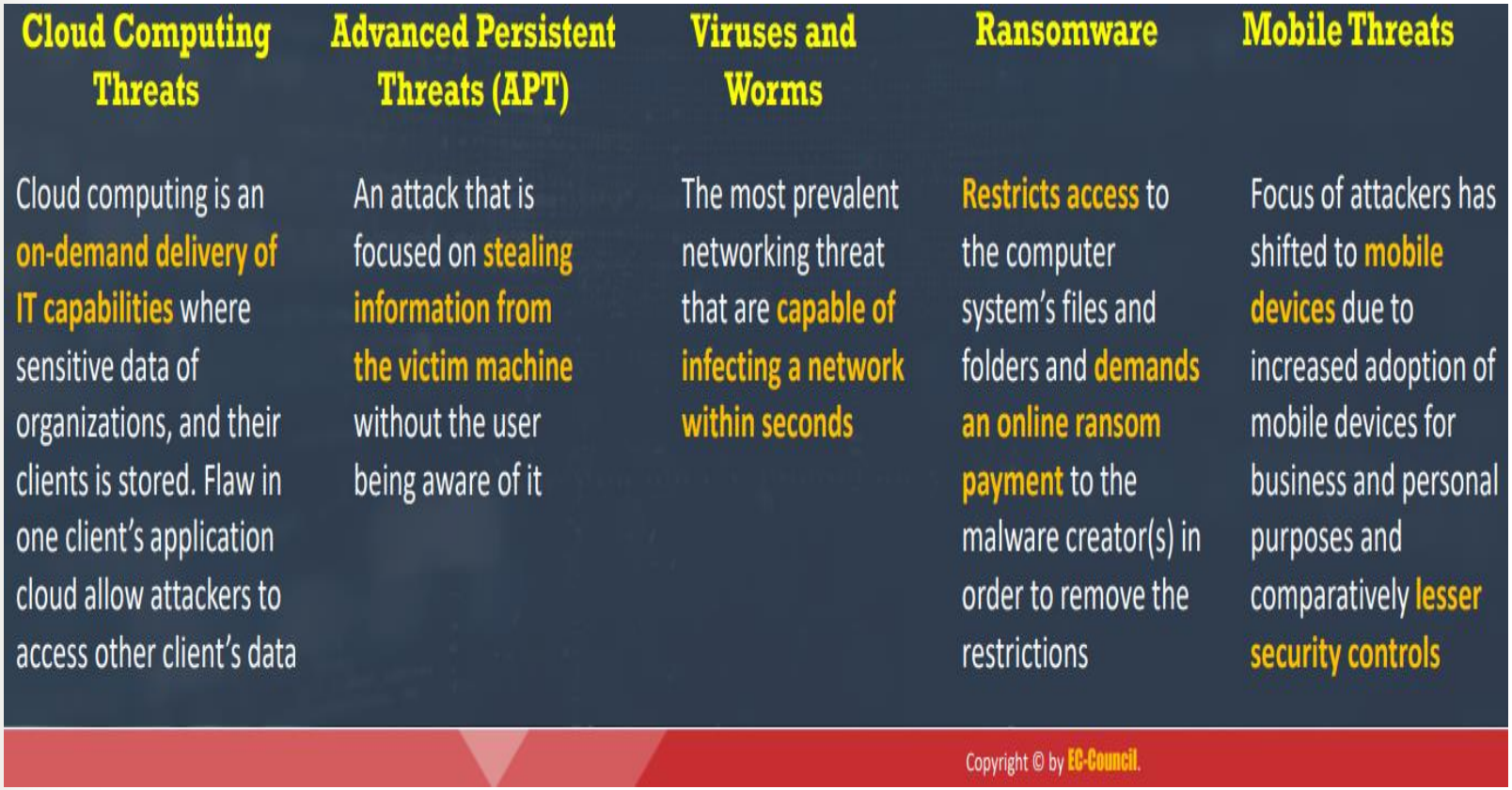
Attackers try various tools and attack techniques to **exploit vulnerabilities** in a computer system or its security policy and controls in order to fulfil their motives

Motives behind information security attacks

- ✓ Disrupting business continuity
- ✓ Stealing information and manipulating data
- ✓ Creating fear and chaos by disrupting critical infrastructures
- ✓ Causing financial loss to the target
- ✓ Damaging the reputation of the target

Information Security Attack Vector

Below is a list of information security attack vectors through which an attacker can gain access to a computer or network server to deliver a payload or seek a malicious outcome.



Cloud Computing Threats

- Cloud computing threats involve risks related to data stored in cloud environments.
- Example: Account hijacking allows attackers to gain unauthorized access to cloud accounts.
- This can happen through phishing or weak passwords, leading to manipulation or theft of sensitive data. A notable incident involved the Capital One breach, where a former employee exploited vulnerabilities to access millions of customer records.
- Cloud computing threats, especially account hijacking, pose significant risks to data integrity and security.

Advanced Persistent Threats (APTs)

- APTs are targeted cyberattacks where intruders remain undetected for extended periods.
- Example: Attackers often use spear phishing to infiltrate high-value targets like government agencies.
- Once inside, they can steal sensitive information over time, using advanced malware to maintain access. This stealthy approach can lead to major data breaches before detection.
- APTs represent a serious risk due to their stealthy nature and potential for extensive data theft.

Viruses and Worms

- Viruses and worms are self-replicating malware that spreads across systems.
- Example: A virus may attach to a legitimate file, while a worm spreads independently through networks.
- Both can disrupt operations by corrupting files and stealing data, making them difficult to control once they infiltrate a system.
- Viruses and worms pose significant threats due to their ability to replicate and spread rapidly.

Ransomware

- Ransomware encrypts files on a victim's system, demanding payment for decryption.
- Example: The WannaCry attack in 2017 affected hundreds of thousands of computers globally.
- Ransomware can cripple organizations by denying access to critical data until a ransom is paid, with no guarantee of recovery afterward.
- Ransomware poses a severe threat, leading to operational disruptions and financial losses.

Mobile Threats

- Mobile threats target mobile devices with risks like malware and insecure applications.
- Example: Mobile banking malware can steal credentials from users' banking apps.
- As mobile devices become essential for personal and business use, they attract attackers who exploit insecure apps and vulnerabilities.
- Mobile threats are significant due to the increasing reliance on mobile devices, heightening the risk of data breaches.

Information Security Attack Vector

Botnet	Insider Attack	Phishing	Web Application Threats	IoT Threats
A huge **network of the compromised systems** used by an intruder to perform various network attacks	An **attack performed on a corporate network** or on a single computer by an **entrusted person (insider)** who has authorized access to the network	The practice of **sending an illegitimate email** falsely claiming to be from a **legitimate site** in an attempt to **acquire a user's personal or account information**	Attackers target web applications to steal credentials, set up phishing site, or **acquire private information** to threaten the performance of the website and hamper its security	• IoT devices include many software applications that are used to **access the device remotely** • Flaws in the IoT devices allows attackers access into the device remotely and perform various attacks

Botnet

- A botnet is a network of compromised devices controlled by an attacker, used for large-scale cyberattacks.
- Example: The Mirai botnet infected thousands of IoT devices to launch a massive DDoS attack on DNS provider Dyn in 2016.
- Botnets can send spam, steal data, or execute DDoS attacks, leveraging the scale and anonymity of numerous infected devices.
- Botnets facilitate large-scale cyberattacks and other malicious activities through networks of compromised devices.

Insider Attack

- Insider attacks occur when individuals within an organization misuse their access to compromise security.
- Example: A disgruntled employee steals sensitive data before leaving to sell it to competitors.
- Insiders can bypass security measures due to their legitimate access, leading to significant data breaches and operational damage.
- Insider attacks pose serious risks because insiders can exploit their access to harm the organization.

Phishing

- Phishing is a social engineering attack that deceives individuals into providing sensitive information or downloading malware.
- Example: An employee receives a fake email from IT asking them to reset their password via a fraudulent link.
- Phishing exploits trust and urgency, leading to credential theft and unauthorized access. Employee education and email filtering are essential defenses.
- Phishing remains a prevalent threat due to its effectiveness in tricking individuals into revealing sensitive information.

Web Application Threats

- Web application threats involve exploiting vulnerabilities in web applications to gain unauthorized access or disrupt services.
- Example: SQL injection attacks manipulate database queries through input fields, exposing sensitive data.
- Poor coding practices can lead to vulnerabilities that attackers exploit. Regular security assessments are crucial for mitigation.
- Web application threats pose serious risks by exploiting software vulnerabilities, potentially leading to significant data breaches.

IoT Threats

- IoT threats involve vulnerabilities in Internet of Things devices that attackers can exploit.
- An attacker exploits weak security in smart home devices to gain access to a home network.
- Explanation: Many IoT devices lack adequate security, making them attractive targets for cybercriminals who can use them as entry points for broader attacks.
- IoT threats are significant due to the vulnerabilities in connected devices that can compromise network security.

Skills required for Ethical Hacking

- Programming & SQL
- Network
- Continuous Learning
- Communication Skills
- Know-how of:
  - Hacking tools to simplify the process of identifying and exploiting weaknesses in system
  - Use of internet and search engines to gather information
  - Basic commands of different OS (Linux, Windows, MAC)
  - Contribute to hacking forums (develop open source programs, answer questions, thank other contributors)

Programming Skills

- Example: Proficiency in languages like Python, Java, and C++ is crucial.
- Programming knowledge allows ethical hackers to write scripts, understand software vulnerabilities, and develop security tools. This foundational skill is vital for navigating complex systems and analyzing code.
- Writing programs as an ethical hacker will help you to automate many tasks which would usually take lots of time to complete.
- Writing programs can also help you identify and exploit programming errors in applications that you will be targeting.

Networking Skills

- Example: Understanding protocols like TCP/IP, DNS, and DHCP.

- A strong grasp of networking concepts helps ethical hackers identify potential security threats within interconnected systems. This knowledge is critical for exploiting network vulnerabilities.

Continuous Learning

- Example: Staying updated with the latest cybersecurity trends and technologies.
- The cybersecurity landscape is constantly evolving, making it essential for ethical hackers to engage in ongoing education and training.

Communication Skills

- Example: Ability to explain technical findings to non-technical stakeholders.
- Explanation: Effective communication is crucial for reporting vulnerabilities and recommending solutions clearly and concisely.

SR NO.	COMPUTER LANGUAGES	DESCRIPTION	PLATFORM	PURPOSE
1	**HTML**	Language used to write web pages.	*Cross platform	**Web hacking** Login forms and other data entry methods on the web use HTML forms to get data. Being able to write and interpret HTML, makes it easy for you to identify and exploit weaknesses in the code.
2	JavaScript	Client side scripting language	*Cross platform	**Web Hacking** JavaScript code is executed on the client browse. You can use it to read saved cookies and perform cross site scripting etc.
3	PHP	Server side scripting language	*Cross platform	**Web Hacking** PHP is one of the most used web programming languages. It is used to process HTML forms and performs other custom tasks. You could write a custom application in PHP that modifies settings on a web server and makes the server vulnerable to attacks.
4	SQL	Language used to communicate with database	*Cross platform	**Web Hacking** Using SQL injection, to by-pass web application login algorithms that are weak, delete data from the database, etc.

** Cross platform means programs developed using the particular language can be deployed on different operating systems such as Windows, Linux based, MAC etc.*

https://www.guru99.com/skills-required-become-ethical-hacker.html

SR NO.	COMPUTER LANGUAGES	DESCRIPTION	PLATFORM	PURPOSE
5	Python Ruby Bash Perl	High level programming languages	*Cross platform	**Building tools & scripts** They come in handy when you need to develop automation tools and scripts. The knowledge gained can also be used in understand and customization the already available tools.
6	C & C++	Low Level Programming	*Cross platform	**Writing exploits, shell codes, etc.** They come in handy when you need to write your own shell codes, exploits, root kits or understanding and expanding on existing ones.
7	Java CSharp Visual Basic VBScript	Other languages	Java & CSharp are *cross platform. Visual Basic is specific to Windows	**Other uses** The usefulness of these languages depends on your scenario.

** Cross platform means programs developed using the particular language can be deployed on different operating systems such as Windows, Linux based, MAC etc.*

https://www.guru99.com/skills-required-become-ethical-hacker.html

Application Security

Courtesy: https://www.imperva.com/learn/application-security/application-security/

https://www.mend.io/resources/blog/

Application Architecture

- Application Architecture is a blueprint or structure of your system that provides a guide to how you assemble software applications and how each of those apps interacts with one another to meet a client's needs.

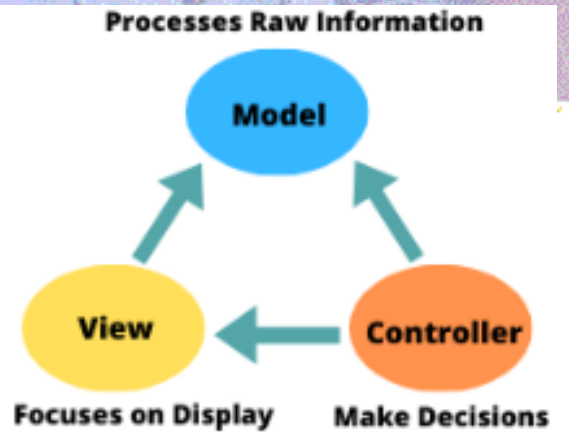
- This structure comprises **software modules** and all their components, systems and the various interactions among them.

- Application architecture can help you define how your software interacts with databases and middleware to ensure your application can **scale** to meet increasing business demands and user requirements while maintaining stable processes.

Key Components of Application Architecture

- **Presentation Layer**:
  - Manages the user interface (UI) and user interactions.
  - Example: Web pages, mobile app screens.
- **Business Logic Layer**:
  - Handles the application's core functionality, rules, and workflows.
  - Example: Processing orders in an e-commerce app.
- **Data Layer**:
  - Manages data storage, retrieval, and database interactions.
  - Example: Storing user profiles in a database.

Types of Application Architecture



- Model-View-Controller (MVC) Architecture:
  - The Model-View-Controller (MVC) architectural pattern is a way of breaking an application or to precisely separate the logic of the code, into three parts: the model, the view, and the controller
  - **Model:** This part manages the data and business logic on your site. Its role is to retrieve the raw information from the database, organize, and assemble it so that it can be processed by the controller.
  - **View:** This part focuses on the display. It is here that the data recovered by the model will be used to present them to the user.
  - **Controller:** This part manages the logic of the code that makes decisions. When the user interacts with the view, the request is processed by the controller. It waits for the user to interact with the view to retrieve the request. Thus, it is the controller that will define the display logic, and display the next view on the screen.

Example: MVC Architecture

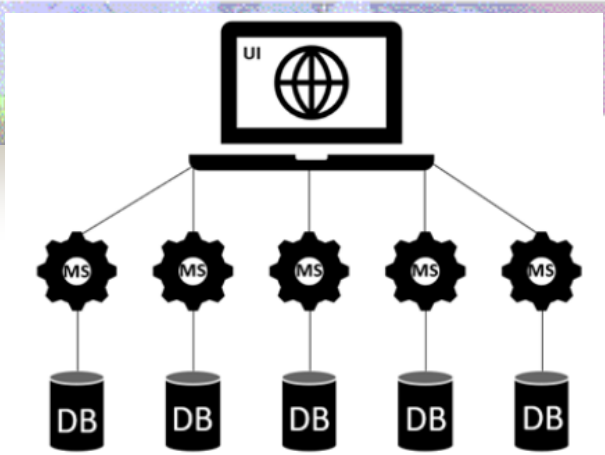
- MODEL: In a shopping app, the `Product` class (with attributes like name, price, etc.) is part of the Model.
- VIEW: A webpage or app screen showing a list of products.
- CONTROLLER: If a user clicks "Add to Cart," the Controller updates the cart in the Model and refreshes the View.

How they work together

- **User interacts with the View** (e.g., clicks a button).
- **View sends the input to the Controller**.
- **Controller processes the input** and updates the Model.
- **Model notifies the View** about the changes.
- **View updates itself** to reflect the new data.

- This separation of concerns makes the code easier to manage, test, and scale.

Types of Application Architecture

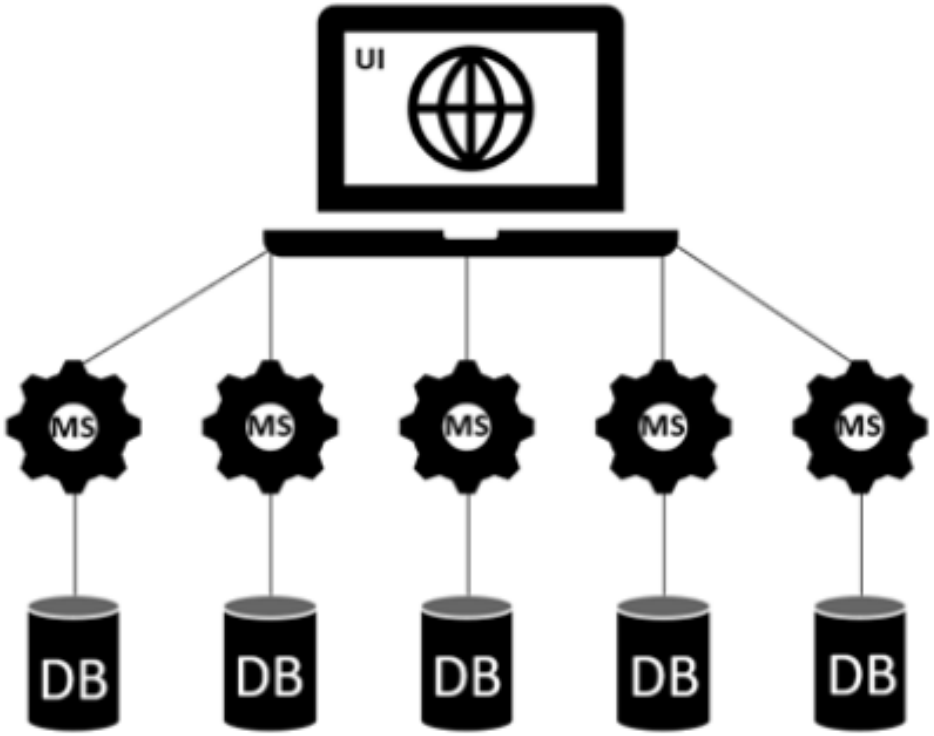


Independent microservices working with their own database to execute functionality requested by the user interface.

- Microservices Architecture:
  - Microservices can be defined as an improvement, a kind of refinement, of what we know as service-oriented architecture (SOA).
  - In this architecture, a large application is made in the form of small monofunctional modules. Each microservice is autonomous.
  - Microservices do not share a data layer. Each has its own database and load balancer. So that each of these services can be deployed, adjusted, and redeployed individually without jeopardizing the integrity of an application.
  - As a result, you will only need to change one or more separate services instead of having to redeploy entire applications.

Types of Application Architecture

- Microservices Architecture:



Independent microservices working with their own database to execute functionality requested by the user interface.

How Microservices work

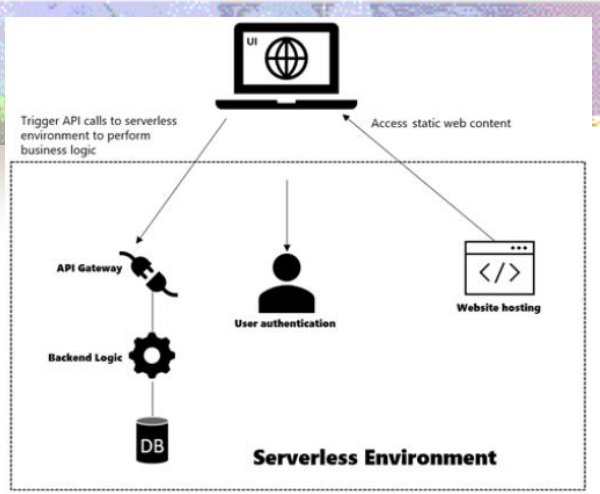
- **Each service performs a specific task**:
  - Example: In an e-commerce app:
    - Product Service: Manages product catalog.
    - User Service: Handles user accounts.
    - Order Service: Processes orders.
- **Services communicate over APIs**:
  - If the user places an order, the Order Service may call the Product Service to verify stock and the Payment Service to process payment.

Examples of companies using Microservices

- **Netflix**: Scales its video streaming services to millions of users using microservices.
- **Amazon**: Uses microservices to power its e-commerce platform.
- **Uber**: Manages ride requests, payment processing, and mapping as separate services.

Microservices Architecture is ideal for large, complex, and scalable applications but requires careful design and robust tooling to manage the inherent complexities.

Types of Application Architecture



Different components of a serverless framework interacting with the UI function.

- Serverless Architecture:
  - Serverless means an organization does not need to invest in or maintain physical hardware. Instead, you rely on a trusted third-party to manage the maintenance of the physical infrastructure, including the server, network, storage, etc.
  - This approach lets your organization develop applications without needing to manage the underlying infrastructure.
  - Serverless includes two different perspectives:
    - **Function as a Service (FaaS):** An evolved model that allows developers to run code module (functions) of an application in real-time, without getting concerned about the backend infrastructure or system requirements.
    - **Backend as a Service (BaaS):** A model where the entire backend (database, storage, etc.) of a system is handled independently and offered as a service. This usually involves outsourcing backend services to a third-party for maintenance and management, leaving your organization to focus on developing your core functions.

How Serverless Architecture works

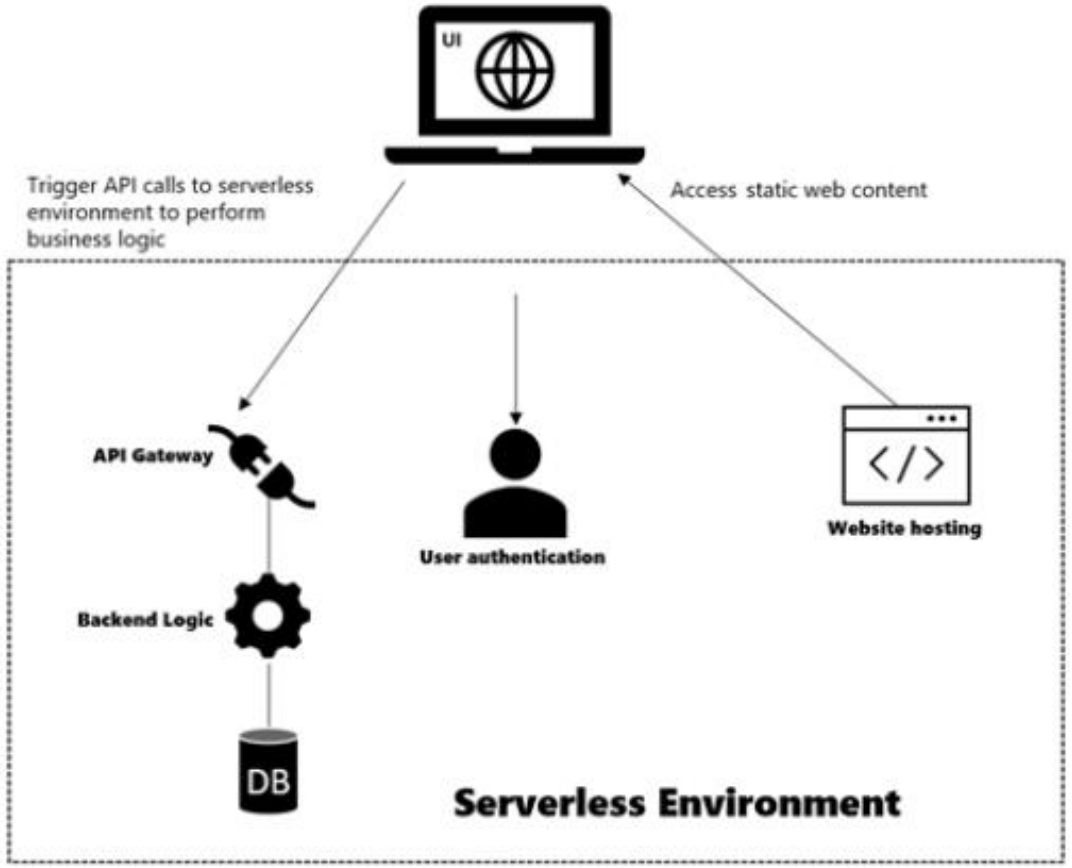
- Developers write **functions** (small pieces of code) to handle specific tasks.
- These functions are deployed to a cloud provider (e.g., AWS Lambda, Azure Functions, Google Cloud Functions).
- When an event occurs (e.g., a user submits a form), the cloud provider triggers the appropriate function.
- The function executes, performs the task, and returns a result.

Examples of Serverless Applications

- **Netflix**: Uses serverless for video encoding and monitoring.
- **Slack**: Processes events like notifications and file uploads using serverless functions.
- **Airbnb**: Manages image processing workflows with serverless.
- Serverless architecture is ideal for applications with unpredictable workloads, rapid development needs, or event-driven requirements, but it requires careful planning to address its limitations.

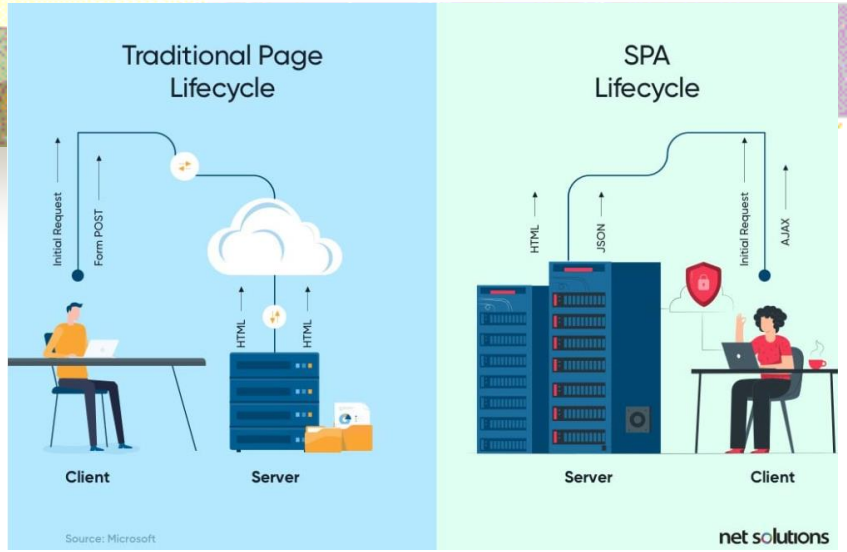
Types of Application Architecture

- Serverless Architecture:



Different components of a serverless framework interacting with the UI function.

Types of Application Architecture



- Single Page Architecture:
- Web application design model where all the necessary resources (HTML, CSS, JavaScript) are loaded once, and subsequent interactions dynamically update the page without requiring a full page reload from the server.
- A SPA application is a single page that continuously interacts with the user by dynamically rewriting the current page rather than loading entire new pages from a server. Trello, Facebook, Gmail, and Twitter are a few single page app examples.
- When you send a request to visit a web page, the browser sends a request to the server and gets an HTML file in return. With a SPA, the server only sends an HTML file on the first request; it sends data known as JSON on subsequent requests.

How SPA work?

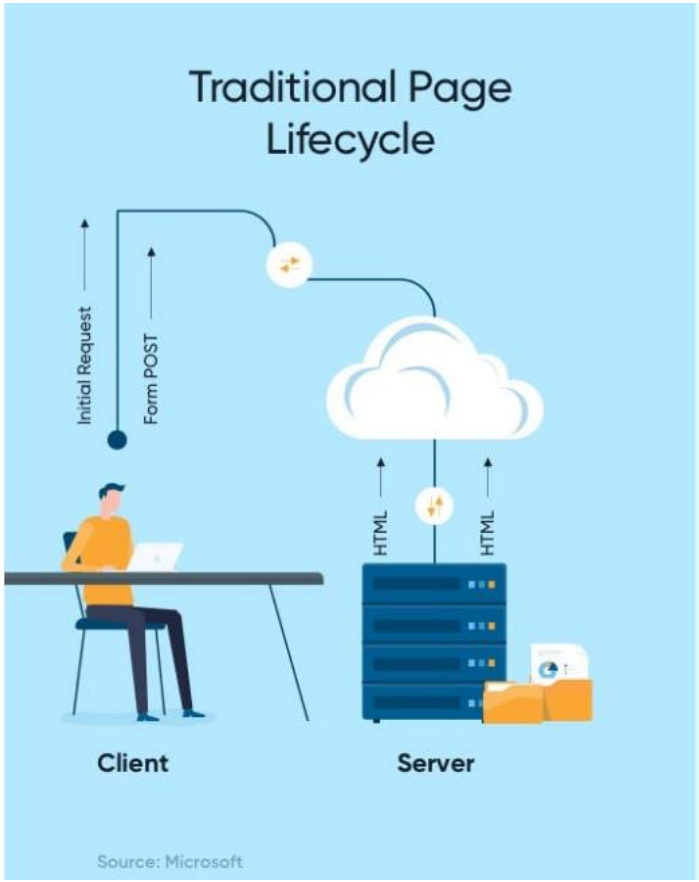
- **Initial Load**:
- When the user accesses the application, the server sends a single HTML file along with the necessary JavaScript and CSS files.

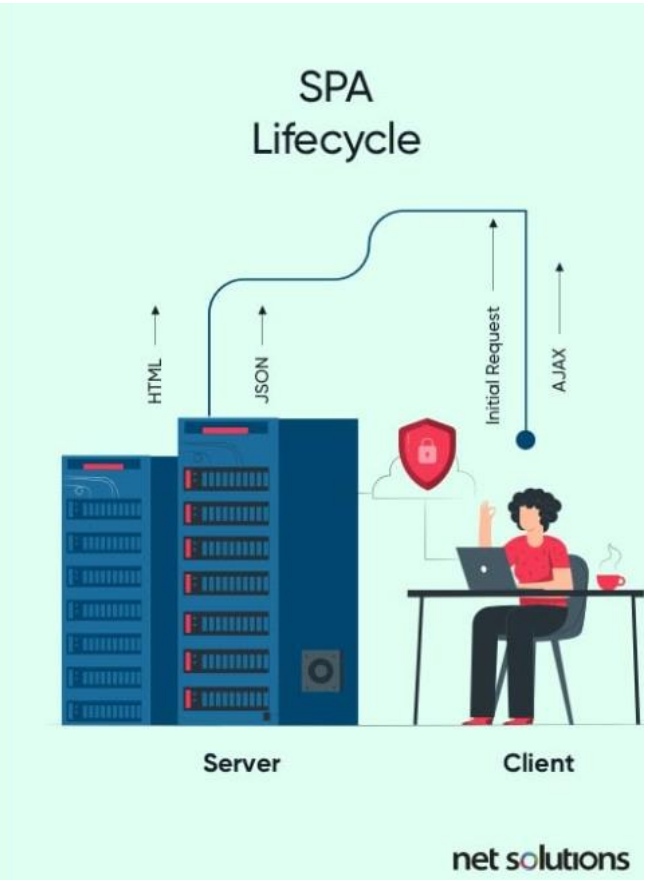
- **Subsequent Interactions**:
- User interactions trigger JavaScript to fetch data asynchronously (e.g., via AJAX or Fetch API) and update the page dynamically.

- **Routing**:
- JavaScript frameworks manage routing (URLs) on the client side, ensuring navigation feels like moving between pages.

Types of Application Architecture

Single Page Architecture:





Examples of SPAs

- **Gmail**:
  - Loads once, and switching between emails is fast and seamless.
- **Google Maps**:
  - Interactions like zooming or searching don’t reload the page.
- **Facebook**:
  - Feeds and notifications update dynamically without page reloads.
- **Trello**:
  - Task management happens in real-time without refreshing.

Trends in Web Application Architecture

- **Web Applications Initially:** consist of a combination of server-side scripts, HTML markup, and styling information in a single file.
- Need to separate concerns lead to the wide use of the **Model-View-Controller (MVC)** design pattern to improve the organization of the application, simplify its creation, and increase maintainability.
- However, very large applications still struggled with staying organized due to the large number of resources and business rules that they had to manage. **Microservices** are a technique that promise to reduce the burden that the server is carrying while providing the array of services associated with large business applications. This is accomplished by separating the server's data and business logic into smaller web services that are then combined to deliver the required features to the user.
- **Single page applications** allow sophisticated applications to be created that are both feature-rich and responsive. It can redraw any part of the UI without requiring a server roundtrip to retrieve HTML. The page does not automatically reload during user interaction with the application or transfer control to another page. Instead, state changes occur via JavaScript using templates and DOM manipulation.

What is Application Security?

- Application security aims to protect software application code and data against cyber threats. You can and should apply application security during all phases of development, including design, development, and deployment.
- Here are several ways to promote application security throughout the software development lifecycle (SDLC):
  - Introduce security standards and tools during design and application development phases. For example, include vulnerability scanning during early development.
  - Implement security procedures and systems to protect applications in production environments. For example, perform continuous security testing.
  - Implement strong authentication for applications that contain sensitive data or are mission critical.
  - Use security systems such as firewalls, web application firewalls (WAF), and intrusion prevention systems (IPS).

Types of Applications does a Modern Organization Need to Secure

- Web Application Security
  - Web applications must accept connections from clients over insecure networks. This exposes them to a range of vulnerabilities.
  - The most severe and common vulnerabilities are documented by the Open Web Application Security Project (OWASP), in the form of the OWASP Top 10.
- API Security
- Cloud Native Application Security

Types of Application Security Testing

- Black Box Security Testing
- White Box Security Testing
- Gray Box Security Testing

Black Box Testing

- Black box security testing focuses on the security of an application while examining it from the outside, rather than testing the code from inside the application.

White Box Testing

- White box is a type of software testing that assesses an application's internal working structure and identifies its potential design loopholes. The term "white box" is used because of the possibility to see through the program's outer covering (or box) into its inner structure. It's also called glass box testing, code-based testing, transparent box testing, open box testing, or clear box testing.

- In this type of testing, the tester has full-disclosure of the application's internal configurations, including source code, IP addresses, diagrams, and network protocols. White box testing evaluates the target system's internal structure—from a developer perspective.

Gray Box Testing

- Gray box testing is a blend of black box and white box testing. Gray box testing is a good way of finding security flaws in programs. It can assist in discovering bugs or exploits due to incorrect code structure or incorrect use of applications.

- A gray box tester takes the code-targeted approach of white box testing and merges it with the various approaches of black box testing like functional testing and regression testing. The tester assesses both the software's internal workings and its user interface.

Application Security Tools and Solutions

- Web Application Firewall (WAF)
- Runtime Application Self-Protection (RASP)
- Software Composition Analysis (SCA)
- Static Application Security Testing (SAST)
- Dynamic Application Security Testing (DAST)
- Interactive Application Security Testing (IAST)
- Mobile Application Security Testing (MAST)
- Cloud-Native Application Protection Platforms (CNAPP)

Web Application Firewall (WAF)

- A WAF monitors and filters HTTP traffic that passes between a web application and the Internet. WAF technology does not cover all threats but can work alongside a suite of security tools to create a holistic defense against various attack vectors.
- In the open systems interconnection (OSI) model, WAF serves as a protocol layer seven defense that helps protect web applications against attacks like cross-site-scripting (XSS), cross-site forgery, SQL injection, and file inclusion.
- Unlike a proxy server that protects the identity of client machines through an intermediary, a WAF works like a reverse proxy that protects the server from exposure. The WAF serves as a shield that stands in front of a web application and protects it from the Internet—clients pass through the WAF before they can reach the server.

Runtime Application Self-Protection (RASP)

- RASP technology can analyze user behavior and application traffic at runtime. It aims to help detect and prevent cyber threats by achieving visibility into application source code and analyzing vulnerabilities and weaknesses.

- RASP tools can identify security weaknesses that have already been exploited, terminate these sessions, and issue alerts to provide active protection.

Software Composition Analysis (SCA)

- SCA tools create an inventory of third-party open source and commercial components used within software products. It helps learn which components and versions are actively used and identify severe security vulnerabilities affecting these components.

- Organizations use SCA tools to find third-party components that may contain security vulnerabilities.

Static Application Security Testing (SAST)

- SAST is known as a “white-box” testing method that tests source code and related dependencies statically, early in the software development lifecycle (SDLC), to identify flaws and vulnerabilities in the code that pose a security threat.
- SAST enables developers to detect security flaws or weaknesses in their custom source code. The objective is either to comply with a requirement or regulation (for example, **Payment Card Industry Data Security Standard** PCI/DSS) or to achieve better understanding of one’s software risk. Understanding security flaws is the first step toward remediating security flaws and thus reducing software risk.

How does SAST work?

- SAST scans organizations' static in-house code at rest, without having to run it. SAST is usually implemented at the coding and testing stages of development, integrating into CI servers and, more recently, into IDEs.
- SAST scans are based on a set of predetermined rules that define the coding errors in the source code that need to be addressed and assessed. SAST scans can be designed to identify some of the most common security vulnerabilities out there, such as SQL injection, input validation, stack buffer overflows, and more.

Dynamic Application Security Testing (DAST)

- DAST is known as a “black-box” testing method that tests the code when it’s running and doesn’t have access to the source code. It is concerned with identifying runtime issues and weaknesses in software and applications. DAST testing is performed later in the SDLC, when software and applications are actually working. A hacker’s rather than a developer’s perspective. DAST is dynamic, because tests as applications run, so it needs a working version of the application for it to perform testing.

How does DAST work?

- DAST works by implementing automated scans that simulate malicious external attacks on an application to identify outcomes that are not part of an expected result set. One example of this is injecting malicious data to uncover common injection flaws. DAST tests all HTTP and HTML access points and also emulates random actions and user behaviors to find vulnerabilities.

Interactive Application Security Testing (IAST)

- IAST is an AST tool designed for modern web and mobile applications that works from within an application to detect and report issues while the application is running. It occurs from within the application server to inspect the compiled source code.

How does IAST work?

- IAST typically is implemented by deploying agents and sensors in the application post build. The agent observes the application's operation and analyzes traffic flow to identify security vulnerabilities. It does this by mapping external signatures or patterns to source code, which allows it to identify more complex vulnerabilities.
- IAST test results are usually reported in real time via a web browser, dashboard, or customized report without adding extra time to the CI/CD (continuous integration/ continuous delivery) pipeline. IAST results can also be combined with other issues tracking tools.


IAST
VS
DAST
VS
SAST
REACTIVE SCANNING
PROACTIVE SCANNING
APPLICATION: STOPPED
APPLICATION: RUNNING
OUTSIDE ACCESS
INSIDE ACCESS
REACTIVE SCANNING
PROACTIVE SCANNING
APPLICATION: STOPPED
APPLICATION: RUNNING
OUTSIDE ACCESS
INSIDE ACCESS
REACTIVE SCANNING
PROACTIVE SCANNING
APPLICATION: STOPPED
APPLICATION: RUNNING
OUTSIDE ACCESS
INSIDE ACCESS

Mobile Application Security Testing (MAST)

- MAST tools employ various techniques to test the security of mobile applications. It involves using static and dynamic analysis and investigating forensic data collected by mobile applications.
- Organizations use MAST tools to check security vulnerabilities and mobile-specific issues, such as jailbreaking, data leakage from mobile devices, and malicious WiFi networks.

Cloud native application protection platform (CNAPP)

- Provides a centralized control panel for the tools required to protect cloud native applications. It unifies cloud workload protection platform (CWPP) and cloud security posture management (CSPM) with other capabilities.

- CNAPP technology often incorporates identity entitlement management, API discovery and protection, and automation and orchestration security for container orchestration platforms like Kubernetes.